GUÍA PRÁCTICA · NEIRO AGENCY

ECC: un equipo de revisión y seguridad dentro de Claude Code



ECC (Everything Claude Code) es un paquete gratuito que le agrega a Claude Code agentes, skills, comandos y hooks para planear, probar, revisar y auditar la seguridad de lo que construís. Esta guía explica qué trae, cómo instalarlo con control y para qué sirve de verdad, según el propio repositorio.

Marcos Vargas IA · Sistemas · Automatización

LO QUE APRENDERÁS

01	Qué es ECC y qué trae adentro
02	Cómo instalarlo sin llenar Claude de ruido
03	Cómo revisar tus apps con sus agentes y skills
04	Agentes que juegan de atacante: qué hacen y qué no
05	Los costos y riesgos de instalarlo
06	El error más común al instalarlo
07	Cómo lo uso yo

SECCIÓN 01
QUÉ ES Y QUÉ TRAE

Un conjunto de archivos y automatizaciones que le dan método a Claude Code.

ECC es open source con licencia MIT, creado por Affaan Mustafa. No es un modelo ni una app que corre sola: Claude Code carga sus archivos y sus hooks ejecutan código en cada acción. La idea central es que Claude siga un proceso fijo: **planear, probar, implementar, revisar y verificar**. Esto es lo que trae la versión 2.2.3, contado en el repositorio:

PIEZA	CANTIDAD	PARA QUÉ SIRVE
Agentes	71	Subagentes con un rol fijo: planificador, arquitecto, revisor de código, revisor de seguridad, guía de TDD, y revisores por lenguaje (Python, TypeScript, Go, Rust, React…).
Skills	311	Procedimientos que Claude carga solo cuando el tema aplica: pruebas, QA en navegador, auditoría de producción, seguridad, patrones de código.
Comandos	97	Atajos como `/plan`, `/code-review`, `/security-scan`, `/build-fix` y `/learn`. El repo se mueve a "skills primero": `/tdd` y `/e2e` ya son skills (`tdd-workflow`, `e2e-testing`) y sus atajos viejos quedaron en una carpeta opcional.
Hooks	8 eventos	Código que corre antes y después de cada herramienta, al iniciar y cerrar sesión, al compactar contexto y al enviar un prompt.
Reglas	22 carpetas	Estándares comunes y por lenguaje. El plugin no las instala: se copian a mano.

Además trae memoria opcional ("instincts", patrones de tus sesiones con puntaje de confianza) y **AgentShield**, un paquete aparte (repo propio, versión 1.6.0) con 268 reglas que escanea tu configuración de Claude Code. También trae skills para tu stack (Postgres, Redis, Docker, migraciones, despliegue, diseño de APIs, servidores MCP) y agentes como `database-reviewer` y `silent-failure-hunter`.

SECCIÓN 02
CÓMO INSTALARLO

Instalalo una sola vez, por una sola vía y con los hooks bajo control.

1. **Elegí un método.** Plugin nativo: `/plugin marketplace add https://github.com/affaan-m/ECC` y luego `/plugin install ecc@ecc`. O el instalador guiado: `npx ecc-universal@2.2.3 setup` (requiere Node 18+ y Claude Code 2.1+). El README advierte que mezclar métodos duplica skills y hooks.
2. **Controlá los hooks.** El plugin trae dos ajustes: `hooks_enabled` (apaga todos los hooks y deja solo skills y comandos) y `hook_profile` (`minimal`, `standard` o `strict`).
3. **Elegí un perfil, no todo.** Hay perfiles de instalación `minimal` (sin hooks), `core`, `developer` (el estándar para apps), `security`, `research` y `full` (33 módulos). Para revisar seguridad, `security` es el que encaja. Con todo cargado, Claude gasta contexto en instrucciones que no necesita.
4. **Instalá desde el repo oficial.** El propio README avisa que las copias de terceros pueden traer malware y que fijar una versión no es lo mismo que auditar el código.

SECCIÓN 03
REVISAR TUS APPS

Una revisión por capas: código, seguridad, pruebas y producción.

PIEZA DE ECC	QUÉ REVISA
Agente `security-reviewer`	Lista OWASP Top 10: inyección, autenticación, datos sensibles, control de acceso, XSS. Busca secretos escritos en el código, valida la entrada de usuarios y corre `npm audit` sobre dependencias.
Skill `security-review`	Checklist para cuando agregás login, endpoints, pagos, subida de archivos o manejo de secretos.
Skill `verification-loop`	Seis fases: build, tipos, lint, tests con cobertura, búsqueda de seguridad y revisión del diff. Termina en PASS o FAIL.
Skill `browser-qa`	Prueba una versión ya desplegada con un navegador automatizado: errores de consola, rendimiento, formularios, login, capturas y accesibilidad. Termina en SHIP o DO-NOT-SHIP.
Skills `production-audit` y `canary-watch`	Preparación para producción con evidencia local, sin mandar tu repo a un servicio externo, y vigilancia de una URL después de cada despliegue.
Skill `mcp-dependency-review`	Revisa estáticamente tus archivos de MCP buscando paquetes sin versión fija, sin ejecutar los servidores.

SECCIÓN 04
AGENTES ATACANTES

Sí hay agentes que atacan, pero leen tu código y tu configuración: no hacen un pentest en vivo.

- **AgentShield `--opus`:** tres agentes en cadena. El **atacante** (red team) busca vías de ataque y cadenas de varios pasos, el **defensor** (blue team) propone cómo endurecer y el **auditor** prioriza el riesgo. Analiza `CLAUDE.md`, permisos, MCP, hooks y agentes, y califica de A a F. Requiere `ANTHROPIC_API_KEY`: consumo de API aparte.
- **Otras pruebas de AgentShield:** `--injection` (pruebas activas de inyección de prompts), `--sandbox` (ejecuta tus hooks aislados y observa qué hacen), `--taint` (sigue el flujo de datos) y `--supply-chain` (revisa de dónde vienen tus paquetes MCP). `--deep` corre varias juntas.
- **Skill `security-bounty-hunter`:** recorre un repositorio buscando fallas alcanzables desde afuera: SSRF, saltarse la autenticación, inyección SQL o de comandos, recorrido de rutas y XSS.
- **Comando `/santa-loop`:** dos revisores independientes tienen que aprobar el cambio antes de que salga.

> QUÉ NO HACEN
>
> Ninguno ataca los endpoints ni la base de datos en vivo de tu app: AgentShield prueba tu configuración de Claude Code y tus agentes, y el resto lee código. Los agentes de ECC traen una regla que les prohíbe generar contenido de ataque. Para probar datos y contenido de verdad, hacelo solo en un entorno tuyo y con datos falsos. Si tu app lee contenido externo con IA, la skill `agent-security-hardening` parte de que todo texto recuperado es no confiable.

SECCIÓN 05
COSTOS Y RIESGOS

Ejecuta código en tu máquina en cada acción y puede volver a Claude más lento.

- **Hooks en cada herramienta:** son scripts de Node y bash. Revisá qué hacen antes de activarlos, y empezá con `hook_profile` en `minimal`.
- **GateGuard cuesta turnos:** su propio registro de cambios reporta unos 45% más de turnos en un piloto, y que ese piloto no logró medir si mejora los resultados porque todas las pruebas pasaron igual. Es opcional.
- **Datos que salen de tu máquina:** `/astra-review` manda tu diff a OpenAI vía Codex CLI. `--opus` e `--injection` de AgentShield mandan el contenido de tu configuración a la API de Anthropic, o a OrcaRouter si lo elegís (ahí, sin secretos). Los “instincts” se guardan localmente.
- **Un solo mantenedor publica cada semana**, según el propio README. Por eso conviene fijar la versión y revisar lo que se actualiza.

> **OJO: LO QUE PUEDE CAMBIAR O NO ESTÁ CLARO**
>
> El repo se actualiza casi a diario, así que las cantidades de esta guía envejecen rápido. La skill GateGuard dice que su prueba A/B mejoró la calidad, mientras su registro de cambios admite que el piloto no midió efecto. La skill `security-scan` menciona Opus 4.6, pero el README de AgentShield dice que usa Opus 5 por defecto. No pude confirmar qué ataca exactamente `--injection`. Las estrellas (cerca de 279 mil) no prueban seguridad. Revisé manifiestos, hooks, skills y el README de AgentShield, pero no audité cada script.

> **EL ERROR MÁS COMÚN**
>
> Instalar todo en el perfil completo porque está de moda. Las 311 skills y los hooks compiten por el contexto de Claude y por tu tiempo. Empezá con lo mínimo y sumá solo lo que vayas a usar.

> **CÓMO LO USO YO**
>
> Lo uso para revisar mis apps y para tener agentes que intentan atacarlas, así pruebo la seguridad de mis datos y de mis contenidos antes de entregar o publicar.

SIGUIENTE PASO

Instalalo en un proyecto de prueba con los hooks en `minimal` y corré `/security-scan` sobre tu configuración. Corregí lo crítico antes de pasar a revisar el código de una app real.

Importante: las versiones, cantidades y funciones de esta guía se revisaron el 16 de octubre de 2026 en la versión 2.2.3 de ECC, y cambian rápido. Confirmá siempre en el repositorio oficial antes de instalar.

Contenido basado en el repositorio oficial github.com/affaan-m/ECC (README, manifiestos del plugin, perfiles de instalación, registro de cambios, SECURITY.md, skills y agentes citados) y en el README de AgentShield (npm: ecc-agentshield 1.6.0). Preparado por Neiro Agency.